PRODYNA

Meetup

Elegant Monolith

Q1 2026

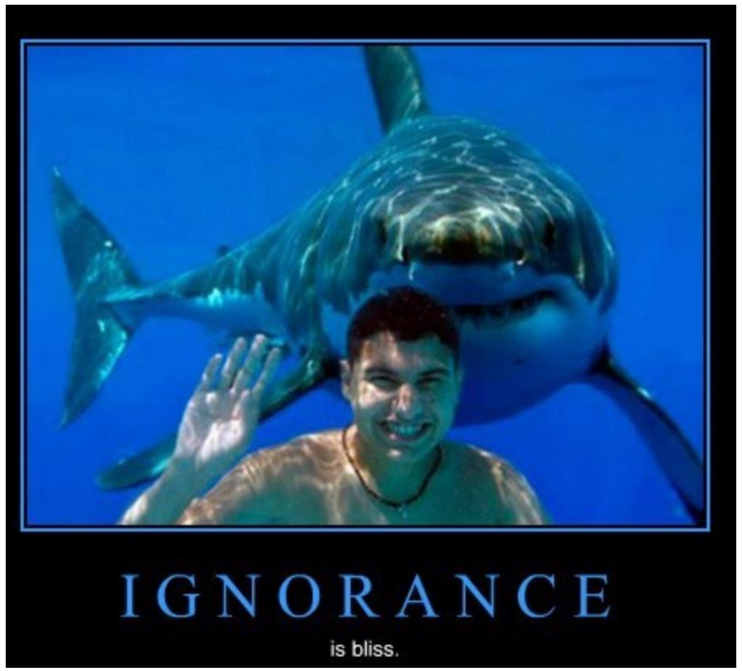
By Georgios Charitakis



Thoughts

- Cloud Era & AI Era
- Who is about?
- Why should you care?





HISTORY of Microservices

PRODYNA

- Early 2000s — SOA era
- Microservices grew out of older ideas like **S**ervice-**O**riented **A**rchitecture (SOA), which promoted decomposing systems into services but often relied on heavyweight ESBs, centralized governance, and large enterprise tools.

HISTORY of Microservices

- 2005–2011 — Proto-microservices
- Companies like Amazon and Netflix began decomposing systems into small, independently deployable components due to scalability needs. They did not call them “microservices” yet

HISTORY of Microservices

- 2011–2014 — The term “microservices” emerges
- The term was popularized by James Lewis and Martin Fowler (2014). Their early definition focused on:
  - Independent deployability
  - Decentralization
  - Polyglot persistence
  - Bounded contexts (explicitly referencing DDD)
- This is where the connection to DDD becomes clearer.

Why microservices?

PRODYNA

- Microservices emerged mainly due to **operational** and **architectural** pressures:
  - scaling web systems
  - deploying independently
  - enabling autonomous teams (Agile)
  - avoiding monolithic coordination

PRODYNA

Microservice design patterns borrowed heavily from DDD:

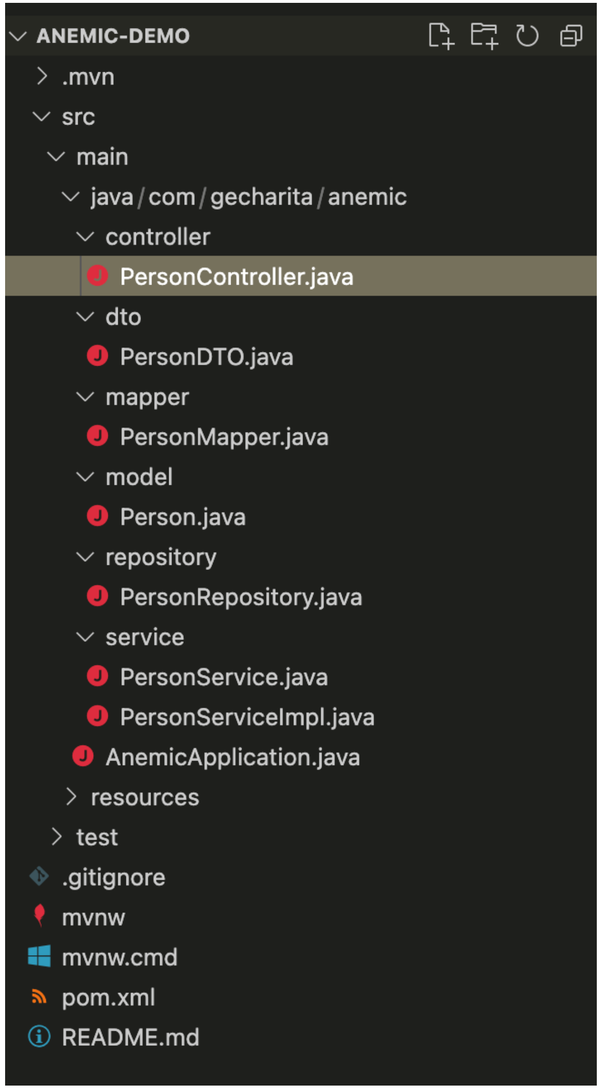
- Bounded Contexts ⟶ service boundaries
- Ubiquitous Language ⟶ service API semantics
- Aggregates ⟶ transactional consistency
- Context Maps ⟶ inter-service communication relationships

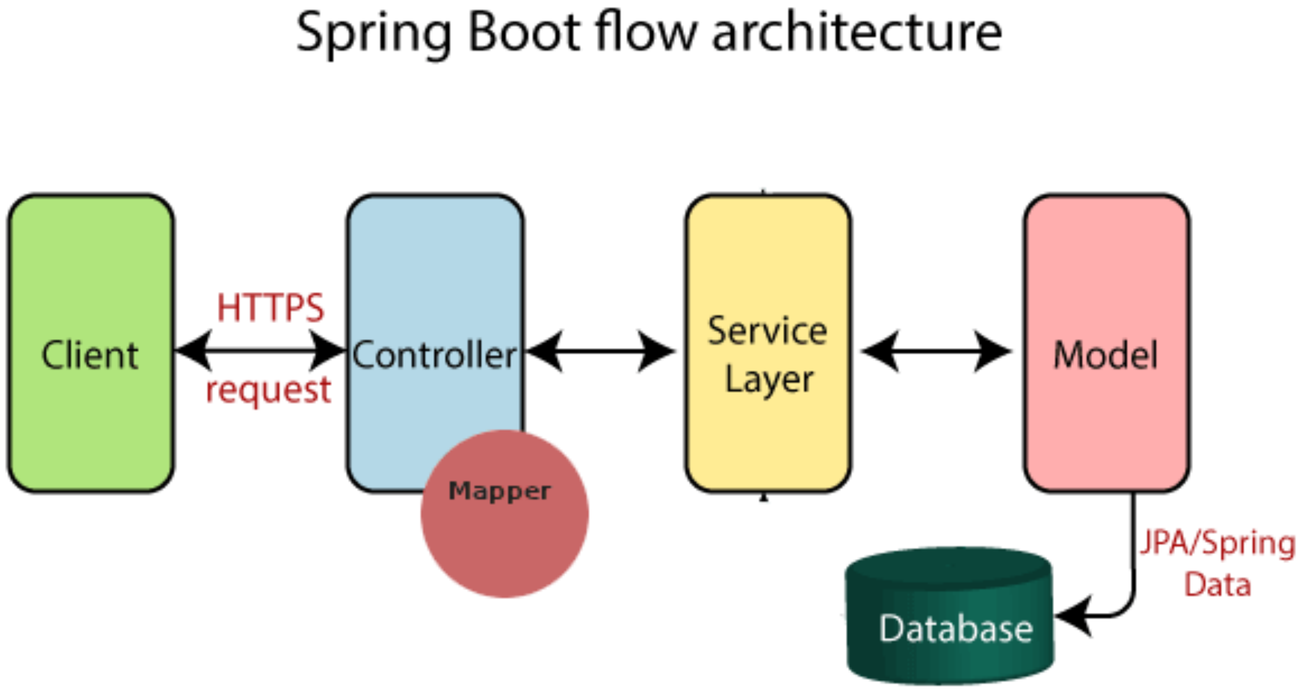
Thus, DDD became a toolkit for designing microservices correctly, but it wasn't the origin.


Gary Bernhardt @garybernhardt · 23 ώ
Απάντηση στον χρήστη @garybernhardt
Consulting service: you bring your startup's 35 microservices to me, I tell you that they'll all fit in a monolith on one dedicated-cpu-2x, you pay me $10,000 for saving you $500,000.

SPRING BOOT STRUCTURE

PRODYNA





Pros & Cons

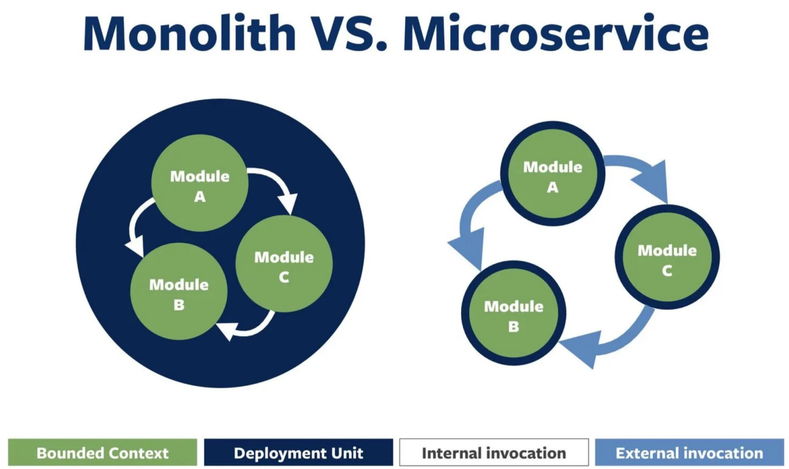
PRODYNA

👍	👎
Simple	Testable
Fast to implement	Extendable
Easier to get in	Complexity can easily increase
Maintainable (if small)	Bulky services
Popular	

Spring Modulith Overview

PRODYNA

- Clear Module Boundaries
- Clear Dependencies between Modules
- Explicit Communication (often vie Events)
- Single Deployable Unit (vs Service Weaver)



Spring Modulith

- IntelliJ Plugin
- Active Project
- Git worktree

DEMO



Spring Modulith Core Features

- Verification of modules
- Integration Tests
- Observability
- Documentation

Spring Modulith vs Multi-Module

- **Spring Modulith** ⟶ modularization *inside the codebase* (via packages + rules)

- **Multi-module** ⟶ modularization *at build level* (via Maven/Gradle subprojects)

Spring Modulith vs All

PRODYNA

	 **Monolith**	 **Spring Modulith**	 **Multi-Module**	 **Microservices**
Code Structure	All in one block	Packages as modules	Separate code per module	Independent services
Dependencies	Free, no restrictions	Strict interfaces per module	Strict via build tools	Via APIs between services
Build & Deploy	Single artifact	Single artifact with module check	Multiple JARs, one fat JAR	Separate deployment per service
Teamwork	Hard to separate	Partially separable	Easy to split by module	Independent teams possible
Complexity	Low	Low-Medium	Medium	High

Thank you



SCAN ME